

Patent Intelligence Project

Research Log and Development Workflow

Contents

Project Objective	3
Stage 1 — Environment Setup	3
Operating System and Shell	3
System Package Setup	3
Project Creation	4
Environment Activation and Python Version	4
Dependency Installation	4
Dependency Verification	4
Compatibility Issue Encountered	5
Resolution	5
Environment Snapshot	5
Successful Loading Code	7
Version Control and Project Structure	7
Stage 1 Summary	8
Stage 2 — Problem and Dataset Exploration	9
Problem Definition	9
How HUPD Was Encountered	9
Datasets Considered	9
Decision: HUPD Selected	10
Open Question: Raw USPTO Data	10
Stage 2 Summary	11
Stage 3 — Understanding HUPD	11
Scope of This Stage	11
What the Dataset Contains	11
Fields Present in the Loaded Sample	12
Still To Be Completed	12
Stage 4 — Loading the HUPD Sample	12
Query Used to Build the Local Sample	12
Local Dataset Storage	13
Loading From Disk	13
Train/Test Naming Convention	13
Dataset Size	14
Notebook Environment	14
Stage 4 Summary	15

Stage 5 — Exploratory Data Analysis	15
Dataset Dimensions	15
Column Types and Missing Values	16
Descriptive Statistics	16
Decision Distribution	17
Current Findings	17
Stage 5 Summary / Current Project Status	18
Next Steps	18
Appendix: Reproducibility Notes	19

Project Objective

This document is a chronological research log of the `patent_intelligence` project. It records what was done, why it was done, what it produced, and what came next — in the order it actually happened, rather than as an idealized plan written after the fact.

The document is organized into stages, and new stages will be appended as the project progresses:

1. Stage 1 — Environment Setup
2. Stage 2 — Problem and Dataset Exploration
3. Stage 3 — Understanding HUPD (*partially completed*)
4. Stage 4 — Loading the HUPD Sample (*completed*)
5. Stage 5 — Exploratory Data Analysis (*initial round completed*)

Stage 1 — Environment Setup

Operating System and Shell

The project is developed on Arch Linux. The terminal prompt shown during development indicates that a Conda `base` environment is active alongside the project's own virtual environment once the latter is activated:

```
(base) [patzer_adi@patzeradi ...]$
```

After activating the project's virtual environment, the prompt becomes:

```
(venv) (base) [patzer_adi@patzeradi patent_intelligence]$
```

Both `(venv)` and `(base)` appear together because Conda's base environment remains active in the shell; however, the Python interpreter and `pip` actually used for the project are those from the project's own `venv`, not from Conda.

System Package Setup

Because the project is developed on Arch Linux, the system Python and `pip` installation were verified using `pacman`:

```
sudo pacman -S python python-pip
```

A package database synchronization / system upgrade was also attempted:

```
sudo pacman -Syyu
```

During this operation, a mirror timeout occurred:

```
error: failed retrieving file 'extra.db' from fastly.mirror.pkgbuild.com :  
Connection timed out after 10003 milliseconds
```

This is recorded as an environmental/network issue encountered during system setup, not as a failure in the project code. The package manager continued into dependency resolution afterward.

Project Creation

The project began with the creation of a dedicated project directory, followed by a Python virtual environment inside it:

```
mkdir patent_intelligence
cd patent_intelligence
python -m venv venv
```

The project directory is located at:

```
/home/patzer_adi/Documents/patent_intelligence/
```

Environment Activation and Python Version

The virtual environment was activated:

```
source venv/bin/activate
```

Python and pip versions were verified with:

```
python --version
pip --version
```

```
Python 3.13.12
pip 26.2.1
```

The pip installation used is located inside the project's own virtual environment:

```
/home/patzer_adi/Documents/patent_intelligence/venv/lib/python3.13/site-packages/pip
```

Dependency Installation

Dependencies were installed into the virtual environment. The core dependencies recorded for this stage were:

```
pip install datasets pandas pyarrow huggingface_hub
```

The purpose of each package:

- **datasets** — loading and processing the HUPD dataset
- **pandas** — tabular data analysis and exploratory data analysis
- **pyarrow** — Feather-format support and dataset-related data handling
- **huggingface_hub** — interaction with the Hugging Face Hub

The most important dependency for this stage was the Hugging Face **datasets** library, since the HUPD dataset is distributed through Hugging Face.

Dependency Verification

The **datasets** package was verified after installation:

```
python -c "import datasets; print(datasets.__version__)"
```

```
3.6.0
```

A basic import check was also performed:

```
python -c "from datasets import load_dataset; print('datasets import OK')"
```

```
datasets import OK
```

An earlier attempt to verify multiple packages at once used the following command:

```
python -c "import datasets, pandas, pyarrow, huggingface_hub; print('Everything works !')"
```

This failed at the shell level, not at the Python level, because Bash interpreted the `!` character as a history-expansion trigger:

```
bash: !': event not found
```

This is recorded as a shell quoting / history-expansion issue rather than a Python package failure.

Compatibility Issue Encountered

The first attempt to load the HUPD dataset failed with the following error:

```
RuntimeError: Dataset scripts are no longer supported,  
but found hupd.py
```

This occurred because the installed version of the `datasets` library had dropped support for the dataset-loading script mechanism that HUPD relies on (HUPD's repository uses a loading script, `hupd.py`).

Resolution

To restore compatibility with the HUPD loading mechanism, the dependency was constrained to versions below 4.0.0:

```
pip install "datasets<4.0.0"
```

The dependency resolver installed `datasets==3.6.0` as the highest version satisfying this constraint. This was not a version chosen directly; it was the result the resolver arrived at once the upper-bound constraint was applied. After this change, the HUPD loading code executed successfully.

Environment Snapshot

The following list records the working environment at the end of Stage 1, obtained via `pip freeze`.

- aiohappyeyeballs — 2.7.1
- aiohttp — 3.14.3
- aiosignal — 1.4.0
- anyio — 4.14.2
- attrs — 26.1.0
- certifi — 2026.7.22
- charset-normalizer — 3.4.9

- click — 8.4.2
- datasets — 3.6.0
- dill — 0.3.8
- filelock — 3.32.2
- frozenlist — 1.8.0
- fsspec — 2025.3.0
- h11 — 0.16.0
- hf-xet — 1.6.0
- httpcore — 1.0.9
- httpx — 0.28.1
- huggingface_hub — 1.26.0
- idna — 3.18
- multidict — 6.7.1
- multiprocessing — 0.70.16
- numpy — 2.5.1
- packaging — 26.3
- pandas — 3.0.5
- propcache — 0.5.2
- pyarrow — 25.0.0
- python-dateutil — 2.9.0.post0
- PyYAML — 6.0.3
- requests — 2.34.2
- six — 1.17.0
- tqdm — 4.70.0
- typing_extensions — 4.16.0
- urllib3 — 2.7.0
- xxhash — 3.8.1
- yarl — 1.24.5

Successful Loading Code

The following code, executed after resolving the compatibility issue, successfully loaded and saved a local copy of the HUPD sample split:

```
from datasets import load_dataset

dataset_dict = load_dataset(
    "HUPD/hupd",
    name="sample",
    data_files="https://huggingface.co/datasets/HUPD/hupd/blob/main/hupd_metadata_2022-02-22.feather",
    icpr_label=None,
    train_filing_start_date="2016-01-01",
    train_filing_end_date="2016-01-21",
    val_filing_start_date="2016-01-22",
    val_filing_end_date="2016-01-31",
)

# Save the dataset locally
dataset_dict.save_to_disk("sample_dataset")

print("Dataset saved successfully!")
print(dataset_dict)
```

Version Control and Project Structure

Once the environment was working and the HUPD sample was loading successfully, the project was pushed to GitHub. The same directory structure used locally is maintained in the remote repository, so that the repository mirrors the working project exactly rather than diverging into a separate “clean” layout.

The project root was kept organized as it grew: source code, the local dataset, notebooks, and generated reports are each kept in their own directory rather than sitting loose at the project root. The current project structure is:

```
patent_intelligence/
|
|-- data/
|   '-- sample_dataset/
|
|-- notebooks/
|   '-- 01_hupd_eda.ipynb
|
|-- src/
|   '-- load_hupd.py
|
|-- reports/
|
|-- requirements.txt
|-- README.md
|-- .gitignore
|
'-- venv/
```

Rationale for each directory:

- **venv/** — the project’s Python environment.

- `data/sample_dataset/` — the local dataset lives here, since this is where datasets belong rather than at the project root.
- `src/load_hupd.py` — the loading script lives here, since it is project/source code rather than an analysis notebook.
- `notebooks/` — numbered analysis notebooks live here, starting with `01_hupd_eda.ipynb`.
- `reports/` — reserved for generated reports, figures, and PDFs produced later in the project.

A `README.md` was added at the project root, since GitHub renders it directly on the repository page; it gives a short entry-point description of the project, while this document (`workflow.tex`, and its compiled `workflow.pdf`) remains the detailed, chronological research log.

Note that `venv/`, `data/`, and `sample_dataset/` are excluded from version control (e.g. via `.gitignore`), since they are either environment-specific or regenerable from code; the directory structure itself, along with the source files, notebooks, and documentation, is what is kept in sync between the local project and GitHub.

Stage 1 Summary

The sequence of events for Stage 1 was as follows:

1. Verify system Python/pip on Arch Linux via `pacman` (mirror timeout encountered and noted as an environmental issue)
2. Create project directory `patent_intelligence/`
3. Create virtual environment `venv/`
4. Activate the virtual environment
5. Confirm Python version (3.13.12) and pip version (26.2.1)
6. Install Python dependencies (`datasets`, `pandas`, `pyarrow`, `huggingface_hub`)
7. Verify dependencies via import checks (one check failed at the shell level due to Bash history expansion, not a package issue)
8. Attempt to load HUPD → compatibility error encountered
9. Constrain dependency: `pip install "datasets<4.0.0"`
10. Resolver installs `datasets==3.6.0`
11. Retry HUPD loading → succeeds
12. Save locally as `sample_dataset/`
13. Push project to GitHub, maintaining the same directory structure, with a dedicated `notebooks/` directory added

Next: understand why HUPD was chosen as the dataset for this project (Stage 2).

Stage 2 — Problem and Dataset Exploration

Problem Definition

The starting problem for this project was not classification or prediction in the usual machine-learning sense (e.g. predicting whether a patent will be granted). It was retrieval: given a new invention described in plain language, find the most semantically similar existing patents, using semantic similarity as a signal for novelty.

The motivation is that keyword-based patent search misses relevant prior art when two patents describe the same invention using different words. For example, a search for “circular rolling device” will not match a patent describing a “wheel-based transportation mechanism” under keyword search, even though the two describe closely related concepts. A semantic (embedding-based) representation is intended to catch this kind of match where keyword search fails.

It is worth noting that HUPD also supports an acceptance-prediction task, since it includes a grant/reject decision field. This was not the project’s target task — it is treated as an incidental field in a dataset that was chosen for other reasons.

How HUPD Was Encountered

HUPD was encountered while reading literature related to semantic patent retrieval and patent embeddings, specifically:

- Suzgun, Melas-Kyriazi, Sarkar, Kominers, and Shieber (NeurIPS 2023), *The Harvard USPTO Patent Dataset* — the paper introducing HUPD.
- Jiang et al. (2024), *Can Large Language Models Generate High-Quality Patent Claims?* (arXiv:2406.19465) — a paper that builds a new claim-generation dataset directly on top of HUPD.

This led to the underlying resources:

- HUPD on Hugging Face: <https://huggingface.co/datasets/HUPD/hupd>
- HUPD GitHub repository: [suzgunmirac/hupd](https://github.com/suzgunmirac/hupd)

The GitHub repository was particularly useful because it showed that the dataset had accompanying documentation and an implementation. This suggested a development strategy: rather than immediately attempting to build the full patent-intelligence system, first reproduce the dataset’s basic loading workflow, then build iteratively on top of it.

Datasets Considered

Several datasets were encountered during this exploration. The table below distinguishes what was actually learned about each one from what was inferred or is still an open question — it does not claim that every alternative was fully investigated or ruled out.

Dataset	What we learned	Current status
CLEF-IP 2011	Retrieval-oriented benchmark (556,750 patents) built around a fixed query set and relevance judgments, rather than a general-purpose corpus.	Potentially useful for evaluation

Dataset	What we learned	Current status
Raw USPTO Bulk Data	Original source data (unstructured XML across years). Using it would mean building our own acquisition and parsing pipeline before any embedding work could start.	Open question — discuss with mentor (see Section)
PatentMatch	Provides claim/prior-art pairs from European search reports, rather than being a large browsable corpus to search over.	Potential future evaluation resource
DAPFAM	A newer (2025) domain-aware, family-level dataset. Encountered during exploration; not yet investigated in depth.	Not selected for initial implementation
HUPD	Structured patent applications with rich text fields, accessible via Hugging Face, and accompanied by documentation and a reference implementation.	Selected for initial development (see Section)

Decision: HUPD Selected

HUPD was not selected because every other dataset considered was proven unsuitable — several of them (CLEF-IP, PatentMatch) may still be useful later for evaluation, and others (DAPFAM, Raw USPTO) simply were not investigated deeply enough to rule in or out with confidence. HUPD was selected because it gave a practical starting point: a large, clean, pre-structured corpus — 34 fields per patent application, with full text sections already split into abstract, claims, background, summary, and description — that could be loaded and used for embedding work immediately, without first building custom parsing or evaluation infrastructure.

This decision applies to the current development phase. It reflects a choice to use HUPD so the semantic retrieval pipeline could be developed iteratively, not a final judgment that HUPD is the permanent or only data source for the project.

Open Question: Raw USPTO Data

Whether the final system should independently acquire and process raw USPTO data, rather than relying on HUPD, has not been decided. This question has not yet been raised with the project mentor and remains open.

A possible future pathway for raw USPTO acquisition, documented here as an unimplemented option, is as follows:

1. Acquire patent/application records
2. Parse XML / source formats
3. Extract metadata
4. Extract text fields:
 - title
 - abstract
 - claims
 - description

- classifications
5. Normalize schema across years
 6. Store locally
 7. Build retrieval corpus

Status: potential future route — not implemented.

Stage 2 Summary

The sequence of events for Stage 2 was as follows:

1. Define the problem: semantic patent retrieval / prior-art discovery
2. Read related literature (Suzgun et al.; Jiang et al.)
3. Encounter HUPD in the literature
4. Investigate HUPD via its paper, its Hugging Face page, and its GitHub repository
5. Compare candidate datasets: CLEF-IP 2011, Raw USPTO Bulk Data, PatentMatch, DAP-FAM, HUPD
6. Select HUPD as the initial development corpus
7. Leave raw USPTO acquisition as an open question for the project mentor

Next: understand the structure of HUPD itself (Stage 3).

Stage 3 — Understanding HUPD

Scope of This Stage

This stage is partially completed. It documents what is currently known about HUPD’s structure directly from working with the loaded sample; a fuller treatment of HUPD’s origin, construction methodology, and the complete 34-field schema (as described in the source paper) is still to be completed.

What the Dataset Contains

HUPD — the Harvard USPTO Patent Dataset — consists of English-language utility patent applications filed with the USPTO. The dataset page is:

<https://huggingface.co/datasets/HUPD/hupd>

The **sample** configuration used in this project corresponds to patent applications filed in January 2016, split by filing date into a training portion and a validation portion (see Section for the exact date ranges used).

Fields Present in the Loaded Sample

The version of the dataset actually loaded into this project (via `load_from_disk`) exposes the following 14 fields:

- `patent_number`
- `decision`
- `title`
- `abstract`
- `claims`
- `background`
- `summary`
- `description`
- `cpc_label`
- `ipc_label`
- `filing_date`
- `patent_issue_date`
- `date_published`
- `examiner_id`

This is the working schema for the project going forward; it is a subset of the full field set described in the original HUPD paper (34 fields), reflecting the specific query used when the sample was requested (Section).

Still To Be Completed

The following items from the original Stage 3 scope have not yet been documented and remain open:

- The full origin and construction methodology of HUPD, and the identity of its creators, beyond the citation already recorded in Stage 2.
- The complete 34-field schema as described in the source paper (as opposed to the 14 fields present in the loaded sample).
- The precise meaning and provenance of the `decision` label categories beyond the three values observed in the data (`PENDING`, `ACCEPTED`, `REJECTED` — see Stage 5).
- The full reasoning, beyond practical convenience, for starting with the January 2016 sample specifically.

Stage 4 — Loading the HUPD Sample

Query Used to Build the Local Sample

The local sample was produced using the loading call already documented in Stage 1 (Section 2.9), with training filings drawn from 2016-01-01 to 2016-01-21 and validation filings drawn from 2016-01-22 to 2016-01-31.

Local Dataset Storage

After the compatibility issue was resolved, the dataset was saved locally. Following the project's directory layout (Section 2.10), the dataset lives under `data/sample_dataset/` and the loading script under `src/load_hupd.py`:

```
patent_intelligence/  
|-- src/load_hupd.py  
|-- data/sample_dataset/  
|-- venv/  
'-- notebooks/
```

The local dataset directory was inspected directly:

```
ls -lh data/sample_dataset
```

```
total 12K  
-rw-r--r-- 1 patzer_adi patzer_adi 35 Aug 5 19:12 dataset_dict.json  
drwxr-xr-x 2 patzer_adi patzer_adi 4.0K Aug 5 19:13 train  
drwxr-xr-x 2 patzer_adi patzer_adi 4.0K Aug 5 19:13 validation
```

Because the dataset is stored locally, it can be reloaded directly from disk rather than downloading it again from Hugging Face.

Loading From Disk

The notebook lives in `notebooks/` while the dataset lives in `data/`, so it is loaded with a relative path from the notebook:

```
from datasets import load_from_disk  
  
dataset = load_from_disk("../data/sample_dataset")  
  
dataset
```

```
DatasetDict({  
  train: Dataset({  
    features: ['patent_number', 'decision', 'title', 'abstract', 'claims',  
              'background', 'summary', 'description', 'cpc_label',  
              'ipc_label', 'filing_date', 'patent_issue_date',  
              'date_published', 'examiner_id'],  
    num_rows: 16153  
  })  
  validation: Dataset({  
    features: ['patent_number', 'decision', 'title', 'abstract', 'claims',  
              'background', 'summary', 'description', 'cpc_label',  
              'ipc_label', 'filing_date', 'patent_issue_date',  
              'date_published', 'examiner_id'],  
    num_rows: 9094  
  })  
})
```

Train/Test Naming Convention

For this project, the HUPD `validation` split is treated as the project's `test` split. This is a project-level naming convention, not a property of HUPD itself:

```
HUPD train      -> project train
HUPD validation -> project test
```

The conversion is performed with:

```
train_df = dataset["train"].to_pandas()
test_df = dataset["validation"].to_pandas()
```

From this point on, the project consistently refers to these DataFrames as `train_df` and `test_df`, with the explicit understanding that `test_df` corresponds to HUPD's original validation split.

Dataset Size

```
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
```

```
Train shape: (16153, 14)
Test shape: (9094, 14)
```

This was independently confirmed with:

```
print("Training samples:", len(train_df))
print("Test samples:", len(test_df))
```

```
Training samples: 16153
Test samples: 9094
```

Summary:

- Training set: 16,153 patent applications
- Test set (HUPD validation): 9,094 patent applications
- Total: 25,247 patent applications
- Number of fields: 14

Notebook Environment

The project uses JupyterLab for exploratory analysis:

```
pip install jupyterlab
```

Matplotlib was also installed into the virtual environment, required for exploratory visualization. The notebook is stored under the project's `notebooks/` directory and follows a numbered naming convention, beginning with `01_hupd_eda.ipynb`. The notebook begins with:

```
# HUPD Exploratory Data Analysis

Exploratory analysis of the Harvard USPTO Patent Dataset (HUPD) sample.

The purpose of this notebook is to understand the structure and characteristics
of the dataset before developing the semantic patent retrieval pipeline.
```

followed by the initial imports:

```
from datasets import load_from_disk
import pandas as pd
import matplotlib.pyplot as plt
```

Two informational (non-blocking) messages appeared during this setup:

```
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets.
Matplotlib is building the font cache; this may take a moment.
```

These are environmental/informational messages and did not prevent the notebook from running.

Stage 4 Summary

1. Confirm the HUPD sample was saved locally under `sample_dataset/`
2. Inspect the local dataset directory (`dataset_dict.json`, `train/`, `validation/`)
3. Load the dataset from disk with `load_from_disk`
4. Adopt the project's train/test naming convention (HUPD `validation` → project `test`)
5. Convert both splits to pandas DataFrames
6. Confirm dataset size: 16,153 training rows, 9,094 test rows, 14 columns
7. Set up JupyterLab and Matplotlib for the exploratory analysis notebook

Next: explore the structure and characteristics of the loaded data (Stage 5).

Stage 5 — Exploratory Data Analysis

Dataset Dimensions

```
print("TRAIN")
print(f"Rows: {train_df.shape[0]}")
print(f"Columns: {train_df.shape[1]}")

print("\nTEST")
print(f"Rows: {test_df.shape[0]}")
print(f"Columns: {test_df.shape[1]}")
```

```
TRAIN
Rows: 16153
Columns: 14
```

```
TEST
Rows: 9094
Columns: 14
```

Column Types and Missing Values

```
pd.DataFrame({
    "column": train_df.columns,
    "dtype": train_df.dtypes.astype(str),
    "missing": train_df.isna().sum().values
})
```

All 14 fields in the training split are represented as strings, and the missing-value check reports zero missing values for every field:

Column	Dtype	Missing
patent_number	str	0
decision	str	0
title	str	0
abstract	str	0
claims	str	0
background	str	0
summary	str	0
description	str	0
cpc_label	str	0
ipc_label	str	0
filing_date	str	0
patent_issue_date	str	0
date_published	str	0
examiner_id	str	0

This is reported as an observation from the training split specifically, and has not been separately verified for the test split. It should also not be read as claiming that no field ever contains an empty string — only that pandas' `isna()` check found no true missing values, since these fields are stored as strings rather than a type where blank/absent values would register as `NaN`.

Descriptive Statistics

```
train_df.describe(include="all").T
```

Field	Count	Unique	Most frequent value	Freq.
patent_number	16153	16153	13261748	1
decision	16153	3	PENDING	7434
title	16153	15286	SEMICONDUCTOR DEVICE	35
abstract	16153	15919	A stand-on physiological sensor (e.g. floormat...)	10
claims	16153	16137	A hybrid oven comprising an oven case; an u...	3
background	16153	14664	(blank)	849
summary	16153	14129	(blank)	1057
description	16153	16125	CROSS REFERENCE TO RELATED APPLICATIONS This a...	4
cpc_label	16153	10323	G06F30416	29

Field	Count	Unique	Most frequent value	Freq.
ipc.label	16153	6698	H04L2906	241
filing_date	16153	20	20160115	1602
patent_issue_date	16153	121	(blank)	9208
date_published	16153	111	20160721	1273
examiner_id	16153	5378	58717.0	55

Notes on this table:

- “(blank)” denotes an empty-string most-frequent value, not a pandas `NaN` — consistent with the zero-missing-values result above.
- The `patent_issue_date` row’s most frequent value is blank with a frequency of 9,208, reflecting that a large share of applications in this sample had not yet been issued a patent (consistent with the large `PENDING` count observed below); no further date-completeness statistic is claimed from this row beyond that.

Decision Distribution

```
train_df["decision"].value_counts()
```

```
decision
PENDING      7434
ACCEPTED     6945
REJECTED     1774
Name: count, dtype: int64
```

```
train_df["decision"].value_counts(normalize=True).mul(100).round(2)
```

```
decision
PENDING      46.02
ACCEPTED     43.00
REJECTED     10.98
Name: proportion, dtype: float64
```

Summary of the training-split decision distribution:

- `PENDING`: 7,434 (46.02%)
- `ACCEPTED`: 6,945 (43.00%)
- `REJECTED`: 1,774 (10.98%)

This indicates that the decision labels are not uniformly distributed, with rejected applications representing the smallest class in this sample. No stronger claim about class imbalance handling or downstream modeling implications is made at this stage.

Current Findings

- The loaded sample contains 25,247 patent applications in total (16,153 train, 9,094 test), each with 14 fields.
- All 14 fields are stored as strings, and no field has missing (`NaN`) values in the training split as checked.

- Several free-text fields (**background**, **summary**) are frequently empty strings rather than populated text.
- The **decision** field has three categories in this sample — PENDING, ACCEPTED, REJECTED — and is imbalanced, with REJECTED being the smallest class.

Stage 5 Summary / Current Project Status

At this stage, the following work has been completed:

1. Arch Linux development environment prepared
2. Python virtual environment created
3. Core Python dependencies installed
4. JupyterLab notebook environment prepared
5. HUPD dataset identified and investigated
6. HUPD loading incompatibility with modern **datasets** versions encountered and resolved
7. HUPD sample dataset successfully obtained
8. Dataset saved locally under **sample_dataset/**
9. Local dataset successfully loaded with **load_from_disk()**
10. HUPD **validation** split mapped to the project's **test** terminology
11. Training and test DataFrames created
12. Dataset dimensions, column types, and missing-value status inspected
13. Descriptive statistics inspected
14. Decision-label distribution analyzed

Next Steps

The following EDA areas are planned but not yet executed, and no results for them should be assumed until they are carried out and documented:

- Decision distribution visualization
- IPC classification distribution
- CPC classification distribution
- Patent text length statistics (abstract, claims, description)
- Comparison of text characteristics across decision classes
- Duplicate / near-duplicate analysis
- Temporal distribution
- Preparation of text fields for semantic retrieval

Appendix: Reproducibility Notes

To reproduce the environment used through Stage 5:

```
sudo pacman -S python python-pip
mkdir patent_intelligence
cd patent_intelligence
python -m venv venv
source venv/bin/activate
pip install "datasets<4.0.0"
pip install pandas pyarrow huggingface_hub matplotlib jupyterlab
```

This should result in Python 3.13.12, pip 26.2.1, and `datasets==3.6.0` being installed, along with the remaining dependencies listed in the Stage 1 environment snapshot (Section 2.9).

The HUPD sample used throughout contains 25,247 records across the two loaded splits:

```
train:      16,153
validation: 9,094   (referred to as "test" in this project)
```

The local dataset should be loaded from `sample_dataset/` using `load_from_disk()` rather than repeatedly re-downloading it from Hugging Face.